

SMART INDIA HACKATHON 2026

Problem Statement Proposal

Predictive Analytics System for Early Detection of Land Acquisition Delays

An AI/ML-Powered Decision Support Platform for Proactive Infrastructure Governance

Team Name	Kiran
Project Name	BhuNirvighna AI
Problem Statement ID	26017
Organization	Ministry of Rural Development
Department	Department of Land Resources (DoLR)
Category	Software
Theme	Smart Automation

Prepared for Internal Hackathon Screening & Grand Finale Submission

1. PROBLEM STATEMENT — SIMPLIFIED EXPLANATION

Building roads, railways, power plants, and other infrastructure in India requires acquiring land first. This process is often unpredictable — a project can move smoothly, or it can get stuck for months or years due to legal disputes, delayed compensation, incomplete paperwork, or resettlement issues. Today, government departments usually discover a project is "stuck" only after the delay has already happened, at which point the damage — cost overruns, missed timelines, public frustration — is already done.

In one line: we want to build a system that looks at a land acquisition project's data and tells officials, in advance, *"this project is at high risk of getting delayed, and here is why"* — so they can act before it actually stalls, instead of finding out after.

Why this matters

- Land acquisition delays are one of the single biggest causes of infrastructure cost and time overruns in India.
- Currently, monitoring is **reactive** — dashboards report what has already gone wrong, not what is about to.
- A predictive, explainable early-warning system shifts governance from reactive reporting to proactive intervention.

2. PROPOSED SOLUTION

2.1 Detailed Explanation of the Proposed Solution

We propose **BhuNirvighna AI** — an AI-powered predictive analytics and decision-support platform that ingests historical and ongoing land acquisition project data and continuously computes a **Delay Risk Score** for every project in the pipeline. The system is built around four tightly integrated components:

- **Predictive Engine:** A machine learning model (Random Forest / XGBoost) trained on project attributes — project type, land area, number of affected families, compensation status, legal disputes, rehabilitation progress, and historical departmental performance — to estimate the probability of delay at each stage of the acquisition lifecycle.
- **Explainability Layer:** Using SHAP (SHapley Additive exPlanations), the system doesn't just output a score — it explains *which specific factors* (e.g., "pending compensation disbursement", "ongoing legal dispute") are driving the risk for that particular project, in plain language.
- **Decision Support Dashboard:** An interactive dashboard for policymakers and district administrators showing risk-ranked project lists, GIS-based risk heatmaps by district/state, delay trend analysis, and comparative performance across departments.
- **Feedback & Continuous Learning Loop:** As projects are resolved or new outcomes are recorded, the model is periodically retrained, so prediction accuracy improves over time rather than staying static.

2.2 How It Addresses the Problem

Current Gap	How BhuNirvighna AI Solves It
No early-warning mechanism — delays are noticed only after they occur.	Risk score computed at project intake and updated continuously — flags risk weeks/months in advance.

Current Gap	How BhuNirvighna AI Solves It
Monitoring is manual and reactive (status reports, not predictions).	Automated, always-on prediction pipeline running on live project data.
No clarity on <i>*why*</i> a project is stuck — officials react to symptoms, not root causes.	SHAP-based explainability surfaces the top contributing factors per project.
Delay data is siloed across departments/states with no unified view.	Centralized dashboard with district/state-wise comparative analytics and GIS visualization.
No structured way to prioritize limited administrative bandwidth.	Risk-based prioritization ranks projects so interventions target the highest-impact cases first.

2.3 Innovation and Uniqueness

- **Explainability-first design:** Most predictive dashboards stop at a risk score. We treat explainability (SHAP) as a first-class feature, not an afterthought — because an unexplained "high risk" label is not actionable for a government administrator; a labelled root cause is.
- **Lifecycle-stage prediction:** Rather than a single static risk label, the model predicts delay probability *at different stages* of the acquisition process (notification → compensation → possession → rehabilitation), so interventions can be stage-specific.
- **Recommendation engine layered on explainability:** SHAP outputs are mapped to human-readable, actionable recommendations (e.g., high weight on 'compensation delay' → recommend expediting disbursement approval), closing the loop from insight to action.
- **Built for continuous improvement:** The system is designed from day one to retrain as new project outcomes arrive, avoiding the common failure mode of static, quickly-outdated government ML tools.

3. TECHNOLOGIES TO BE USED

The stack is deliberately chosen to be lean, well-supported, and fast to implement and demo within a hackathon timeframe, while remaining production-credible for a government deployment context.

Layer	Technology	Purpose
Programming Language	Python 3.x	Core language for data processing, ML, and backend logic
Data Handling	Pandas, NumPy	Cleaning, transforming, and engineering features from raw project data
ML Framework	scikit-learn, XGBoost	Training the Random Forest / Gradient Boosted classifier for delay risk prediction
Explainable AI	SHAP (SHapley Additive exPlanations)	Per-project, per-feature explanation of risk drivers
Dashboard / Frontend	Streamlit	Rapid, interactive web dashboard for administrators and policymakers
Data Visualization	Plotly, Matplotlib	Delay trend charts, risk distribution, comparative analytics
GIS Visualization	Folium (streamlit-folium)	District/state-level choropleth risk maps on an interactive map
Database	PostgreSQL / SQLite (prototype)	Structured storage of project records and prediction history
Model Persistence	joblib / pickle	Saving and reloading trained models for the continuous learning loop
Deployment	Streamlit Community Cloud / Docker	Hosting the working prototype for live demonstration
Version Control	Git & GitHub	Source control and collaborative development

Hardware: No specialised hardware is required for the prototype — a standard laptop/development machine is sufficient, since the models are lightweight, tabular ML models rather than deep learning models requiring GPUs. For a production deployment, standard government cloud infrastructure (e.g., NIC/MeitY empanelled cloud, or state data centres) would host the backend and database.

4. METHODOLOGY AND PROCESS FOR IMPLEMENTATION

4.1 System Architecture & Data Flow

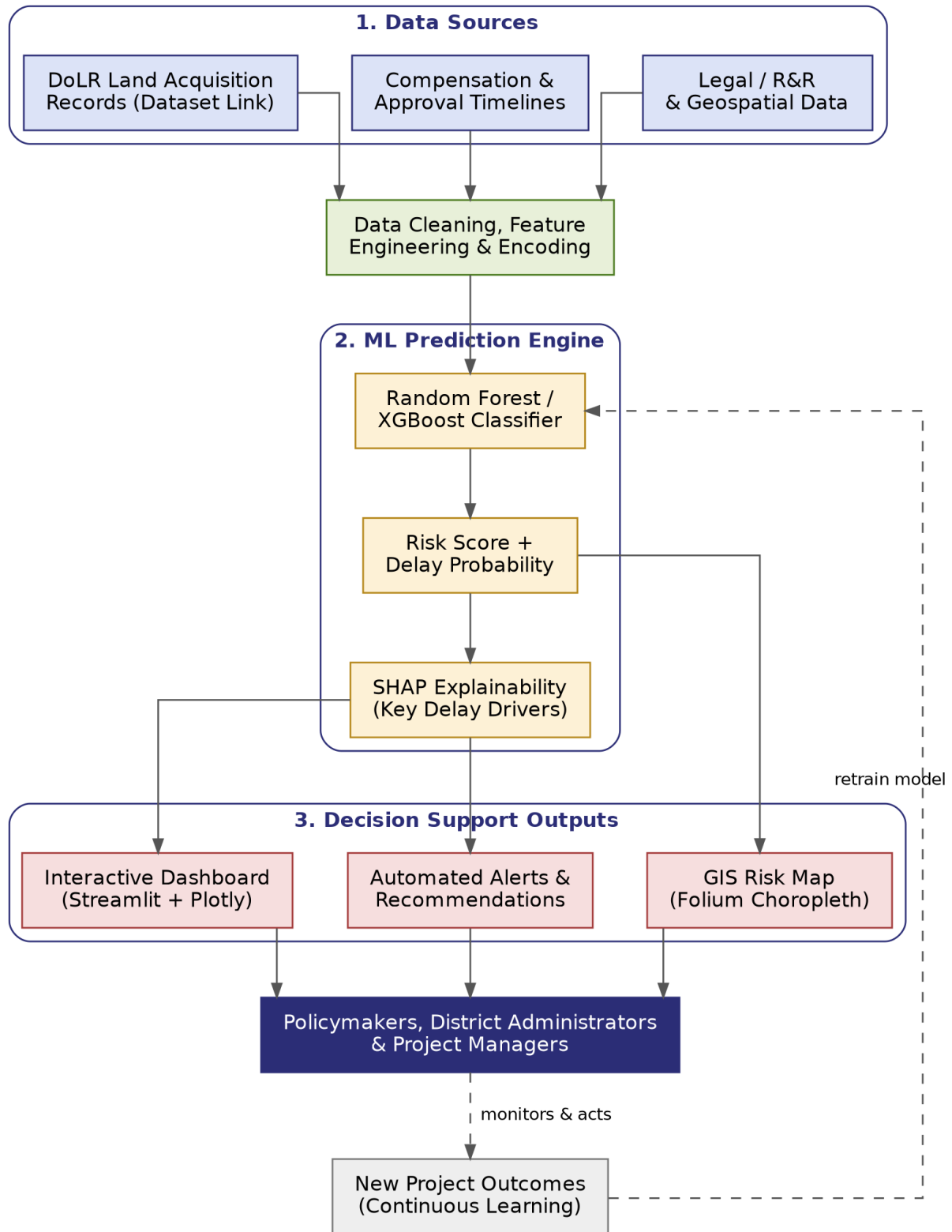


Figure 1: End-to-end system architecture and data flow of the proposed platform

4.2 Step-by-Step Implementation Process

Step 1 — Data Acquisition & Preparation

Ingest the DoLR-provided dataset (project records, compensation status, timelines, legal disputes). Clean missing values, encode categorical fields (project type, state, department), and engineer derived features such as 'days since notification' and 'compensation disbursement ratio'.

Step 2 — Exploratory Data Analysis (EDA)

Analyse historical delay patterns across project types, states, and departments to validate that meaningful, learnable signal exists in the data before model building begins.

Step 3 — Model Training

Train and compare baseline (Logistic Regression) against ensemble models (Random Forest, XGBoost) using stratified train/test splits and k-fold cross-validation to avoid overfitting.

Step 4 — Explainability Integration

Apply SHAP to the trained model to generate global feature-importance summaries and per-project local explanations of predicted risk.

Step 5 — Dashboard Development

Build the Streamlit interface: risk-ranked project table, GIS choropleth map (Folium), trend charts (Plotly), and a project drill-down view showing SHAP-based explanations and recommended actions.

Step 6 — Alerting & Recommendation Logic

Implement rule-based mapping from top SHAP drivers to plain-language corrective-action recommendations, and a simple threshold-based alert mechanism for newly high-risk projects.

Step 7 — Testing & Validation

Validate model outputs against held-out historical cases, sanity-check explanations with domain logic, and stress-test the dashboard with the full dataset.

Step 8 — Packaging for Demonstration

Prepare the working prototype, demo script, architecture diagram, and presentation for the evaluation round.

5. FEASIBILITY ANALYSIS

5.1 Technical Feasibility

- The core problem is a well-understood **tabular classification/regression task** — a domain where scikit-learn/XGBoost are mature, reliable, and require no exotic infrastructure.
- SHAP is a widely adopted, well-documented library with direct compatibility with tree-based models, keeping integration risk low.
- Streamlit + Folium + Plotly together allow a fully interactive dashboard to be built and deployed within hours, not days — ideal for hackathon timelines.
- An official dataset has been provided by DoLR, removing the biggest technical risk (lack of real, representative data) upfront.

5.2 Operational & Resource Feasibility

- No specialised hardware (e.g., GPUs) is required — the solution can be built and demonstrated on standard laptops.
- The team's existing skillset (Python, scikit-learn, Streamlit, dashboarding) directly matches the stack required, minimizing the learning curve during the build window.
- The modular architecture allows scope to be adjusted — core prediction + explainability + dashboard can be built first, with GIS, alerts, and APIs layered in as time permits.

5.3 Feasibility Scorecard

Dimension	Assessment	Rationale
Data Availability	High	Official DoLR dataset link provided for the problem statement
Modeling Complexity	Low–Medium	Standard tabular ML; no deep learning / CV / NLP dependency
Time-to-Prototype	High	Streamlit + sklearn stack enables a working demo within a 36-hour window
Team Skill Match	High	Directly aligned with existing ML/dashboarding experience
Explainability Requirement	Medium	SHAP has a learning curve but is well-documented and achievable
Full-Scope Requirement (APIs, RBAC, audit trail)	Medium–Low	Achievable only in reduced/mocked form within hackathon time

5.4 Potential Challenges and Risks

Challenge / Risk	Impact	Mitigation Strategy
Real dataset may have significant missing/inconsistent fields	Model accuracy and validity of insights	Perform thorough EDA first; apply robust imputation; clearly document data limitations in the final report
SHAP visualizations and API are unfamiliar to the team	Time lost mid-hackathon on debugging	Learn and prototype SHAP on a small sample dataset before the hackathon begins
Full requirement list (APIs, RBAC, audit trails, real-time alerts) is large for the time available	Risk of a shallow, incomplete demo across too many features	Prioritize: build a small number of features (prediction + explainability + dashboard) deeply rather than all requirements shallowly; mock remaining components conceptually in the presentation
GIS/choropleth mapping is a new tool for the team	Integration delays	Use streamlit-folium with a pre-built India district GeoJSON to reduce implementation time
Model may reflect biases present in historical administrative data	Unfair or misleading risk scores for certain regions/departments	Perform fairness checks across states/departments; present results with clear confidence caveats, not as ground truth
Judges may question real-world deployability vs. hackathon prototype	Perceived as a toy demo	Clearly present a phased roadmap: prototype → pilot with one state/department → phased national rollout

POTENTIAL IMPACT ON TARGET AUDIENCE

6.1 Who Benefits

Stakeholder	Benefit
Ministry of Rural Development / DoLR	Centralized, data-driven visibility into project health across states and departments
District Administrators / Project Managers	Early, specific warnings on which projects need intervention, and why
Policymakers	Evidence-based prioritization of resources and interventions across regions
Affected Families / Landowners	Faster resolution of compensation and rehabilitation issues due to earlier intervention
Infrastructure Contractors / Implementing Agencies	Reduced uncertainty in project timelines, better planning of resources
Taxpayers / General Public	Reduced public expenditure waste from delayed infrastructure projects

6.2 Benefits of the Solution

Social Benefits

- Faster resolution of compensation and rehabilitation grievances for displaced families.
- Increased transparency in how land acquisition decisions and delays are monitored and acted upon.
- Builds public trust through more accountable, proactive governance.

Economic Benefits

- Reduces cost overruns caused by stalled infrastructure projects.
- Optimizes allocation of limited administrative and financial resources toward high-risk projects.
- Accelerates infrastructure delivery timelines, with downstream economic multiplier effects.

Environmental & Governance Benefits

- Better-planned land acquisition reduces ad-hoc, rushed decisions that can lead to poor environmental site assessment.
- Shifts governance culture from reactive status-reporting to proactive, predictive decision-making — a model that can extend to other infrastructure-monitoring domains beyond land acquisition.

7. PRE-EXISTING SOLUTIONS AND COMPARISON

Land acquisition monitoring in India today relies primarily on administrative MIS (Management Information System) portals and periodic manual status reporting. These systems track *what has happened* but do not predict *what is about to happen*. The table below compares existing approaches with the proposed solution.

Existing Approach	Limitation	How BhuNirvighna AI Differs
State/Central MIS portals for land acquisition status tracking	Purely descriptive — shows current status, not future risk	Predictive risk scoring flags likely delays before they occur
Manual periodic progress reports by field officers	Slow, inconsistent, dependent on individual reporting discipline	Automated, continuously updated risk assessment from structured data
Generic project management dashboards (e.g., spreadsheet-based tracking)	No AI/ML component; no root-cause analysis; not scalable across states	ML-driven, explainable, and designed for cross-state/district comparability
Static delay-cause studies / academic research on land acquisition bottlenecks	One-time analysis; not operational or real-time	Live, continuously retrained system embedded into day-to-day administrative workflow
PRAGATI (PMO's ICT-based multi-modal review platform)	General-purpose project review tool; not specialized for land-acquisition-specific risk prediction or explainability	Domain-specific model tailored to land acquisition risk factors, with explainable, stage-wise delay prediction

Summary: No existing publicly known system in this domain combines (a) predictive risk scoring, (b) explainable AI for root-cause transparency, and (c) an integrated GIS-enabled dashboard specifically for land acquisition delay management. This is the core gap BhuNirvighna AI is designed to fill.

8. REFERENCES AND RESEARCH LINKS

Official Problem Statement & Data

- Smart India Hackathon 2026 — Official Problem Statement Portal: <https://sih.gov.in/sih2026PS>
- PS26017 Dataset & Additional Information (DoLR): https://drive.google.com/drive/folders/1ibmzWpI_nK7aBhQurs22R9kqh9fPAQwC

Technology & Methodology References

- scikit-learn documentation — Ensemble Methods (Random Forest, Gradient Boosting): <https://scikit-learn.org/stable/modules/ensemble.html>
- XGBoost official documentation: <https://xgboost.readthedocs.io/>
- SHAP (SHapley Additive exPlanations) documentation: <https://shap.readthedocs.io/>
- Streamlit documentation: <https://docs.streamlit.io/>
- streamlit-folium (GIS mapping in Streamlit): <https://github.com/randyzwitch/streamlit-folium>

Government / Policy Context

- Right to Fair Compensation and Transparency in Land Acquisition, Rehabilitation and Resettlement Act, 2013 (LARR Act) — governing legal framework for land acquisition in India.

- PRAGATI (Pro-Active Governance and Timely Implementation) — PMO ICT-based project review platform, referenced for comparative context: https://www.pmindia.gov.in/en/government_tr_rec/pragati/
- Department of Land Resources, Ministry of Rural Development: <https://dolr.gov.in/>

This document is a working proposal prepared for Smart India Hackathon 2026 internal screening and presentation purposes.